

Picking up last lecture's cliffhanger

Our logistic-regression cheese detector reports **97% accuracy**. Ship it?

It predicts “good” for every batch. Cheese is 3% bad, so being right 97% of the time means catching **zero** bad batches. **Accuracy looked great and was useless.**

This deck: the metrics that actually tell you whether a classifier is any good — and how to turn its *scores* into *decisions*.

Outline

Accuracy and its problem

Precision, recall, F1

Threshold dependence and the ROC fix

When ROC fails: PR and lift

Practical: choosing the metric

Multi-class evaluation

Threshold tuning

Wrap-up

The cheese factory (our running example)

A factory makes 1000 cheese wheels a day; about **3% are bad** (contaminated or spoiled). A bad wheel reaching customers means recalls, illness, lawsuits.

Positive (rare, $\sim 3\%$): a *bad* batch.

Negative (common, $\sim 97\%$): a *clean* batch.

The model outputs a **score** (risk), not a hard label — more useful, as we'll see.

Same factory as the logistic-regression lecture; now we ask: *how do we know the detector is any good?*

What we need from a metric

1. A number to **compare** candidate models.
2. A way to pick a **threshold** that turns scores into decisions.
3. Respect for the **asymmetric cost** — missing bad cheese $\gg$ over-inspecting clean.

The cascade (each one fixes the previous one's flaw): accuracy $\rightarrow$ precision/recall $\rightarrow$ ROC AUC $\rightarrow$ PR AUC.

Accuracy: the obvious starting point

$$\text{Accuracy} = \frac{\text{correct}}{\text{total}} = \frac{TP + TN}{\text{all}}$$

- ▶ Simple to compute, simple to explain to stakeholders.
- ▶ “What fraction of batches did we classify correctly?”
- ▶ The default metric in many tutorials.

Sounds reasonable — let's see when it stops being useful.

TP	FN
FP	TN

accuracy = shaded diagonal / all

Predict-first: where accuracy breaks

Your factory has 1000 daily batches: 970 clean, 30 contaminated. You ship a model that predicts “clean” for *every single batch* (no real model — just the constant).

Predict: its accuracy? its recall on contaminated batches? would you ship it?

- ▶ **Accuracy:** $970/1000 = 97\%$. Looks excellent.
- ▶ **Recall on positives:** 0%. Catches none of the contaminated batches.
- ▶ **Ship it?** Customers get sick. Lawsuits. Recalls cost millions.

The fundamental problem with accuracy: it weights every error equally. When classes are imbalanced, the rare class becomes invisible. *We need a metric that focuses on the positive class.*

Precision, recall, F1

Accuracy weights every error equally, so the rare 3% that matters vanishes.

Fix: stop counting all correctness and **focus on the positive class**.

The confusion matrix

All classification metrics derive from four counts. Lay them on a 2×2 :

	Pred. Positive	Pred. Negative
Actual Pos.	TP bad cheese flagged	FN bad cheese missed!
Actual Neg.	FP wasted inspection	TN clean cheese passed

In the cheese factory the costs are **asymmetric**: an **FN** (contaminated cheese reaches customers → recall/illness/lawsuit) is the expensive mistake; an **FP** (a clean batch re-inspected) is annoying but cheap.

Heads-up: sklearn's `confusion_matrix` is $[TN, FP; FN, TP]$ (negative class first); pass `labels=[1,0]` for the positive-first layout above.

The same errors statisticians call Type I & Type II

The two mistakes are the classic errors from **hypothesis testing** (here H_0 = “the batch is clean”):

- ▶ **Type I = FP**: reject a true H_0 — flag a clean batch (false alarm).
- ▶ **Type II = FN**: fail to reject a false H_0 — pass a bad batch (a miss).

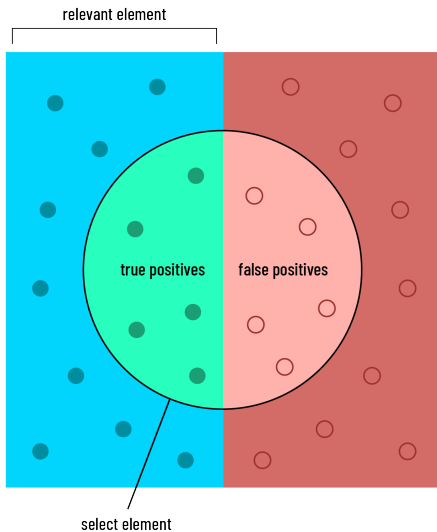
Same 2×2 , two vocabularies. We introduced these in the **Hypothesis Testing** lecture (math/25_stat_hypothesis_testing.qmd).



Figure 3.1 Type I and Type II errors

The classic “pregnant?” picture of Type I vs. Type II.

Precision and recall: of what, out of what?



How many selected items are relevant?

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

F1: when you need a single number

F1 is the *harmonic* mean of precision and recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why harmonic, not arithmetic? The harmonic mean is dominated by the smaller value:

Precision	Recall	Arithmetic mean	Harmonic mean (F1)
1.0	0.0	0.5	0.0
0.8	0.8	0.8	0.8
0.5	0.9	0.7	0.64
0.9	0.5	0.7	0.64

Arithmetic mean lets a 1.0 in one column hide a 0.0 in the other; the harmonic mean refuses — so F1 can't be gamed by an extreme threshold.

F_β generalization: $F_\beta = (1 + \beta^2) \frac{P \cdot R}{\beta^2 P + R}$. Higher β weights recall more; $\beta = 2$ fits the cheese factory (a miss $\gg$ a false alarm).

Worked example: cheese batch inspection

50 batches inspected; 10 actually bad; the model flags 13.

	Pred Bad	Pred Good	Metric	Calc.	Value
Actual Bad (10)	TP = 8	FN = 2	Accuracy	$(8+35)/50$	86%
Actual Good (40)	FP = 5	TN = 35	Recall	$8/(8+2)$	80%
			Precision	$8/(8+5)$	62%
			F1	$2 \cdot .62 \cdot .80 / (.62 + .80)$	70%

Accuracy (86%) hides the 2 missed bad batches; recall (80%) is the operational truth; precision (62%) says 38% of inspections were wasted; F1 (70%) punishes the gap between the two.

Thresholds and the ROC curve

Precision, recall and F1 all shift as you move the cutoff c . Fix: summarize a model across every threshold at once.

Precision and recall come from a threshold

“Flag more $\rightarrow$ recall up, precision down” — that tension **is** the threshold knob. Most classifiers output a *score*, not a label:

$$\text{predict positive} \iff \text{score}(\mathbf{x}) > c.$$

- ▶ **Lower** c ($\rightarrow 0$): flag more $\rightarrow$ higher recall, lower precision (more false alarms).
- ▶ **Higher** c ($\rightarrow 1$): flag fewer $\rightarrow$ higher precision, lower recall (more misses).
- ▶ **Default** $c = 0.5$ **is just one point on a curve** — often not the right one.

So instead of one (P, R) pair, summarize the model over *all* thresholds.

Predict-first: P and R at extreme thresholds

Two extreme cutoffs — what are precision and recall at each?

- ▶ $c = 0$: predict positive for every batch.
- ▶ $c = 1$: predict negative for every batch.

- ▶ $c = 0$: $FN = 0 \Rightarrow$ **recall** = 100%; $FP =$ all negatives $\Rightarrow$ **precision** = **base rate** ($\sim 3\%$).
- ▶ $c = 1$: $TP = 0 \Rightarrow$ **recall** = 0%; precision = 0/0 (sklearn returns 0).

Each of P and R is **trivially gamed** by an extreme threshold. F1 stops that but *still depends on the threshold you picked*. We want a metric that summarizes performance *across all thresholds at once*.

The ROC curve: TPR vs. FPR

Two ratios: **TPR** = recall = $\frac{TP}{TP + FN}$ and **FPR** = $\frac{FP}{FP + TN}$ (the false-alarm rate).

Sweep the cutoff from $c = 1$ (nothing flagged) down to $c = 0$ (all flagged); at each step plot the point (FPR, TPR). The trail is the **ROC curve**.

- ▶ Top-left corner = perfect (TPR = 1 at FPR = 0).
- ▶ Diagonal = random guessing; below it = worse than random (flip the labels).

Let's build one, point by point (next).

Building a ROC curve — the recipe

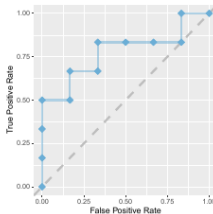
FROM PROBABILITIES TO LABELS: ROC CURVE

Remember: Both probabilistic and scoring classifiers can output classes by thresholding:

$$h(\mathbf{x}) = [\pi(\mathbf{x}) \geq c] \quad \text{or} \quad h(\mathbf{x}) = [f(\mathbf{x}) \geq c_f].$$

To draw a ROC curve:

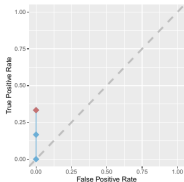
- ❶ Rank test observations on decreasing score.
- ❷ Start with $c = 1$, so we start in $(0, 0)$; we predict everything as negative.
- ❸ Iterate through all possible thresholds c and proceed for each observation x as follows:
 - If x is positive, move TPR $1/n_+$ up, as we have one TP more.
 - If x is negative, move FPR $1/n_-$ right, as we have one FP more.



Step by step: high c first, TPR climbs

DRAWING ROC CURVES

#	Truth	Score
1	Pos	0.95
2	Pos	0.86
3	Pos	0.69
4	Neg	0.65
5	Pos	0.59
6	Neg	0.52
7	Pos	0.51
8	Neg	0.39
9	Neg	0.28
10	Neg	0.18
11	Pos	0.15
12	Neg	0.06

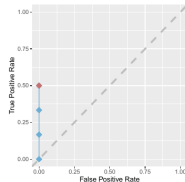


$c = 0.85$
→ TPR = 0.333
→ FPR = 0



DRAWING ROC CURVES

#	Truth	Score
1	Pos	0.95
2	Pos	0.86
3	Pos	0.69
4	Neg	0.65
5	Pos	0.59
6	Neg	0.52
7	Pos	0.51
8	Neg	0.39
9	Neg	0.28
10	Neg	0.18
11	Pos	0.15
12	Neg	0.06



$c = 0.66$
→ TPR = 0.5
→ FPR = 0

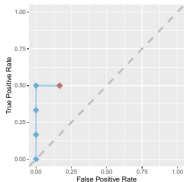


As c drops past the top-ranked points (all real positives here), **TPR steps up** while FPR stays 0.
Source: LMU i2mI, CC BY 4.0.

Step by step: lower c , FPR moves right

DRAWING ROC CURVES

#	Truth	Score
1	Pos	0.95
2	Pos	0.86
3	Pos	0.69
4	Neg	0.65
5	Pos	0.59
6	Neg	0.52
7	Pos	0.51
8	Neg	0.39
9	Neg	0.28
10	Neg	0.18
11	Pos	0.15
12	Neg	0.06

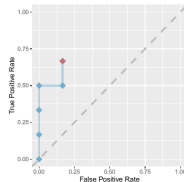


$c = 0.6$
→ TPR = 0.5
→ FPR = 0.167



DRAWING ROC CURVES

#	Truth	Score
1	Pos	0.95
2	Pos	0.86
3	Pos	0.69
4	Neg	0.65
5	Pos	0.59
6	Neg	0.52
7	Pos	0.51
8	Neg	0.39
9	Neg	0.28
10	Neg	0.18
11	Pos	0.15
12	Neg	0.06



$c = 0.55$
→ TPR = 0.667
→ FPR = 0.167



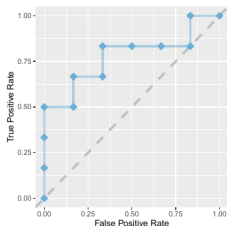
Once negatives start crossing the threshold, **FPR steps right**.

Source: LMU i2ml, CC BY 4.0.

The finished ROC curve

DRAWING ROC CURVES

#	Truth	Score
1	Pos	0.95
2	Pos	0.86
3	Pos	0.69
4	Neg	0.65
5	Pos	0.59
6	Neg	0.52
7	Pos	0.51
8	Neg	0.39
9	Neg	0.28
10	Neg	0.18
11	Pos	0.15
12	Neg	0.06



$c = 0$

→ TPR = 1

→ FPR = 1



At $c = 0$ everything is flagged $\Rightarrow$ (FPR, TPR) = (1, 1).

The staircase joining (0, 0) to (1, 1) is the **ROC curve** — one curve summarizing the model at every threshold.

Source: LMU i2ml, CC BY 4.0.

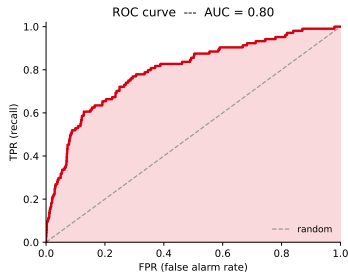
ROC AUC

Summarize the whole ROC curve as one number: the **area under the curve**.

- ▶ $AUC = 1.0$: perfect (curve hits top-left).
- ▶ $AUC = 0.5$: random (curve = diagonal).
- ▶ $AUC = 0.0$: perfectly wrong (flip labels).

Probabilistic meaning. AUC = the probability that a random positive scores higher than a random negative (the Mann-Whitney U statistic) — a real probability, not just an area.

Threshold-free. Compares models on ranking quality without picking a cutoff.



Real cheese model: shaded area = AUC .

Footgun: feed AUC the *scores* (`predict_proba`), not the predicted labels — labels collapse it toward accuracy.

When ROC lies: PR and lift

On rare-positive data the huge negative class hides real differences in ROC.

Fix: measure precision directly — the PR curve.

Predict-first: ROC's blind spot on imbalanced data

Two models, same imbalanced cheese data: 100 bad batches, 10,000 clean.

	TP	FP	FN	TN
Model A	91	10	9	9,990
Model B	91	100	9	9,900

Predict: recall, FPR, precision for each — which would you ship?

	Recall	FPR	Precision
Model A	91%	$10/10,000 = 0.10\%$	$91/101 = 90\%$
Model B	91%	$100/10,000 = 1.0\%$	$91/191 = 48\%$

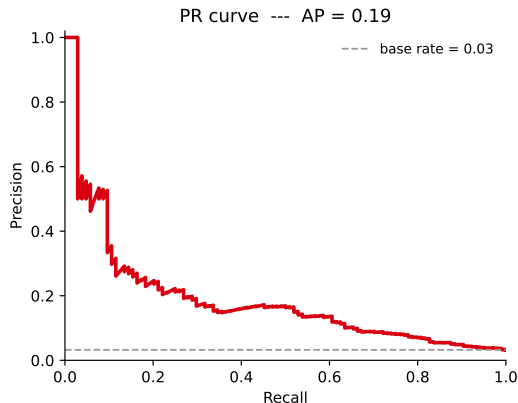
ROC's blind spot: FPR barely moved ($0.1\% \rightarrow 1\%$) — tiny in ROC space — yet precision *crashed* $90\% \rightarrow 48\%$ (half of B's inspections wasted). The huge negative class dominates the FPR denominator and hides the operational difference.

The PR curve

Same threshold sweep, but plot **precision vs. recall** instead of TPR vs. FPR.

- ▶ Top-right = perfect (precision = 1, recall = 1).
- ▶ Top-left = strict threshold (few but high-quality alarms).
- ▶ Bottom-right = loose threshold (many alarms, many false).
- ▶ Baseline: “always positive” has precision = base rate.

Doesn't depend on TN — the TN count never enters precision or recall. That is exactly what makes PR informative when positives are rare.



PR AUC (Average Precision)

Summarize the PR curve as one number — **average precision**, the area under it:

$$AP = \sum_n (R_n - R_{n-1}) \cdot P_n$$

(n indexes the distinct recall steps as you sort by score).

- ▶ $AP = 1.0$: perfect ranking.
- ▶ $AP = \text{base rate}$: random ranking. *Not 0.5!*
- ▶ Higher AP = better ranking, weighted toward high-precision regions.

Sensitive to imbalance, by design. In the A-vs-B example ROC AUC barely changes but PR AUC drops sharply (precision dropped sharply). For rare-class problems, PR AUC matches operational reality.

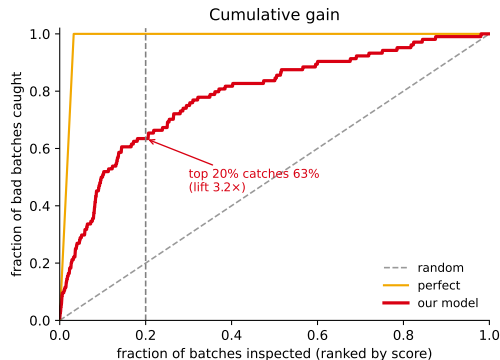
Lift and gain charts: the “top- $k\%$ ” question

Operational constraint: the factory can inspect only the top 20% most-suspicious batches per day. *Which to inspect, and how much better than random?*

Cumulative gain. Rank by score; x = fraction inspected, y = fraction of bad caught. Random = diagonal; perfect = climbs to 100% by the base rate.

Lift = $\frac{\% \text{ bad caught in top } k\%}{k\%}$. Random = 1; lift 5 at the top decile = $5\times$ more bad caught than random for the same budget.

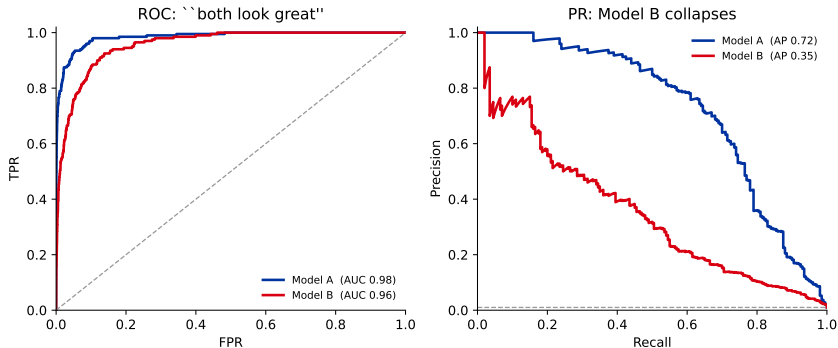
Why it matters. AUCs summarize the whole curve; lift answers the question production actually asks: *given my budget, how much better than random?*



Real cheese model; annotation reads off the lift at the top 20%.

ROC vs. PR side-by-side on imbalanced data

Same imbalanced dataset (1% positive), same two models. Two views:



Same models, two stories. ROC: both near top-left (AUC 0.96–0.98). PR: A holds precision (AP 0.72), B collapses toward baseline (AP 0.35). On imbalanced data *ROC overstates and PR clarifies* — reach for PR when the positive class is rare.

The metric zoo: one matrix, many names

Every metric is a ratio of TP, FP, FN, TN — they split by *what you condition on*:

Given the actual class (row-wise):

- ▶ **TPR** = recall = sensitivity = $\frac{TP}{TP+FN}$
- ▶ **FNR** = miss rate = $\frac{FN}{TP+FN} = 1 - \text{TPR}$
- ▶ **TNR** = specificity = $\frac{TN}{TN+FP}$
- ▶ **FPR** = fall-out = $\frac{FP}{TN+FP} = 1 - \text{TNR}$

Given the prediction (column-wise):

- ▶ **PPV** = precision = $\frac{TP}{TP+FP}$
- ▶ **FDR** = $\frac{FP}{TP+FP} = 1 - \text{PPV}$
- ▶ **NPV** = $\frac{TN}{TN+FN}$
- ▶ **FOR** = $\frac{FN}{TN+FN} = 1 - \text{NPV}$

Overall: accuracy = $\frac{TP+TN}{\text{all}}$; **balanced accuracy** = $\frac{\text{TPR}+\text{TNR}}{2}$ (imbalance-robust); **F1** (harmonic mean of precision & recall); **MCC** $\in [-1, 1]$ (correlation of prediction & truth, 0 = random). ROC plots TPR vs. FPR; PR plots precision vs. recall.

Which metric for which problem?

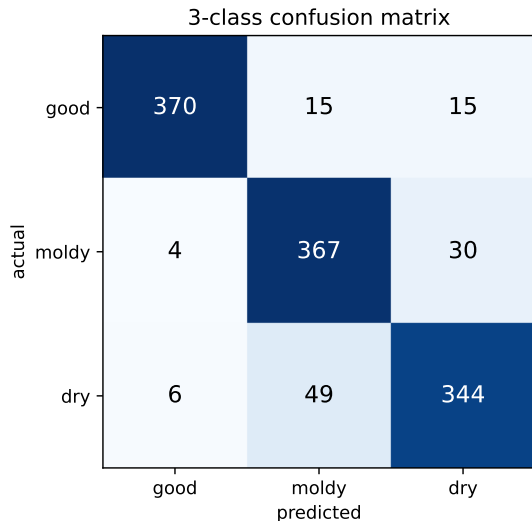
Situation	Use	Why
Balanced classes, single threshold	F1 (or accuracy)	Single number; symmetric.
Balanced, threshold-free	ROC AUC	Compares ranking quality.
Imbalanced, threshold-free	PR AUC (avg. precision)	ROC overstates; PR matches reality.
Bounded inspection budget	Lift / gain at top- k %	Capacity is the real constraint.
FN $\gg$ FP (cheese, fraud, screening)	Recall, with a precision floor	Asymmetric cost.
FP $\gg$ FN (spam, churn calls)	Precision, with a recall floor	Asymmetric cost, other way.
One imbalance-robust number	MCC (Matthews CC)	Symmetric; $\in [-1, 1]$; 0 = random.

Default: report 3–4 metrics — one threshold-free (ROC or PR) plus precision/recall at a chosen threshold.
Calibration is its own lecture.

Beyond two classes

With K classes the confusion matrix grows to $K \times K$, and the binary metrics need an averaging rule. (Cheese defects: good / moldy / dry.)

The $K \times K$ confusion matrix



- ▶ Diagonal = correct; off-diagonal shows *which classes get confused*.
- ▶ Here **moldy** and **dry** bleed into each other (30 and 49); **good** is clean.
- ▶ Per-class precision/recall come from **one-vs-rest**: treat “class k vs. the rest” as a binary problem and read its row/column.

One matrix, K binary views.

Averaging: macro vs. micro vs. weighted

Compute the binary metric one-vs-rest per class, then average. Three ways:

- ▶ **Macro:** average per-class, *equal weight per class*. Best when every class matters regardless of size (“how well on each cheese type?”).
- ▶ **Micro:** pool all $TP/FP/FN$ across classes, then compute. *Equals accuracy* for single-label problems. Best for overall correctness.
- ▶ **Weighted:** per-class, weighted by class support. Reflects the data distribution (the common default in reports).

Multi-class AUC: average per-class one-vs-rest (macro-OVR is standard).

Caveat: with few samples in a class, its per-class F1 is noisy and macro F1 inherits that noise — always report support (samples per class).

Picking the operating point

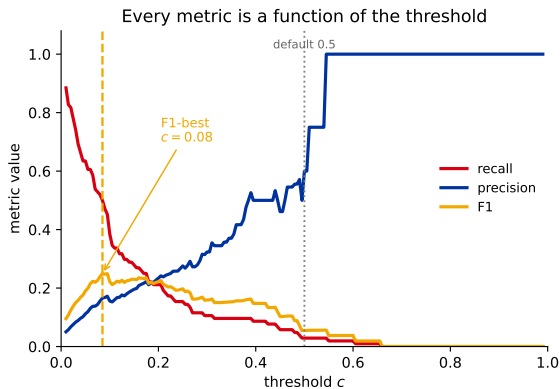
AUC and PR rank models *without* a threshold. To deploy you must choose one cutoff c — and on imbalanced, asymmetric-cost problems 0.5 is almost never it.

Every metric is a function of the threshold

The model outputs scores; the **cutoff** c turns them into decisions. Sweep it:

- ▶ low $c \Rightarrow$ flag more $\Rightarrow$ recall up, precision down;
- ▶ high $c \Rightarrow$ flag fewer $\Rightarrow$ precision up, recall down.

The **default 0.5 is just one point**. Here the F1-best cutoff is ≈ 0.08 — the 3% imbalance pushes it far left of 0.5.



Cost-sensitive thresholding

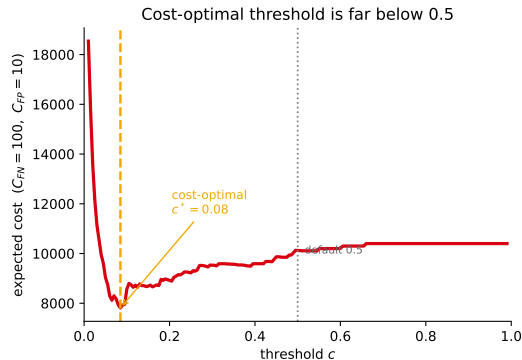
Choose c to minimize *expected cost*, not a symmetric metric:

$$c^* = \arg \min_c C_{FN} FN(c) + C_{FP} FP(c).$$

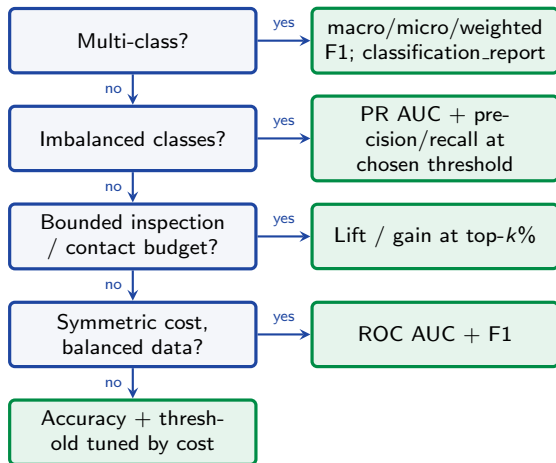
With a miss 10× a false alarm, $c^* \approx 0.08$ here — far below 0.5 (real cheese ratios are larger, pushing it lower still).

Tune on a validation set, never on the test set you report — otherwise the chosen threshold is optimistically biased too.

In sklearn, `TunedThresholdClassifierCV` (≥ 1.5) picks c by cross-validation for any scorer — including a business-cost scorer.



Decision flowchart



Always report multiple metrics. A single number hides the trade-off; the trade-off is the story.

Recap: the cascade

Each metric fixes the previous one's problem:

1. **Accuracy** — simple, fails on imbalance (97% accuracy, 0% recall).
2. **Precision & recall** — focus on the positive class. *But threshold-dependent.*
3. **F1** — one number that punishes a P/R gap. *Still threshold-dependent.*
4. **ROC AUC** — threshold-free ranking quality. *Overstates on imbalance.*
5. **PR AUC** — threshold-free *and* imbalance-sensitive: the rare-class metric.
6. **Lift / gain** — the operational “top- $k\%$ ” question under a budget.

Plus: multi-class averaging (macro/micro/weighted) and **threshold tuning** (the cost-optimal cutoff rarely equals 0.5).

One-line takeaway: report multiple metrics — always one that matches your data's imbalance and your problem's cost structure. **Next:** calibration.